

Homework 2: Multi-Agent Search

Part 1: Minimax Search (9%)

Explain your implementation (4%):

```
class MinimaxAgent(MultiAgentSearchAgent):
    """
    MinimaxAgent :
    使用 Minimax 演算法在搜尋樹中找出 Pacman 的最佳行動。

    - Pacman (agentIndex=0) 是 MAX 玩家，目標是最大化分數。
    - 所有鬼魂 (agentIndex>=1) 是 MIN 玩家，假設會選擇讓 Pacman 最差的動作。
    - 搜尋深度由 self.depth 決定，depth=1 代表 Pacman 走一步 + 所有鬼魂各走一步。
    - 葉節點（終止狀態或達到最大深度）由 self.evaluationFunction 評分。
    - 支援任意數量的鬼魂，每個 depth 層包含一個 MAX 層 (Pacman) 和多個 MIN 層（每隻鬼各一層）。
    """

    # minimax 是核心遞迴函式
    # 根據當前 agentIndex 和 depth 來決定是 MAX 還是 MIN 節點，並返回該節點的分數
    def minimax(self, state, agentIndex, depth):

        # 終止條件1：遊戲已結束（贏或輸），直接回傳評估分數
        if state.isWin() or state.isLose():
            return self.evaluationFunction(state)

        # 終止條件2：輪到 Pacman 且已達到最大搜尋深度，回傳葉節點評估分數
        if agentIndex == 0 and depth == self.depth:
            return self.evaluationFunction(state)

        # 計算下一個行動的 agent：(目前index+1) 對總agent數取餘數，超過就回到 Pacman(0)
        nextAgent = (agentIndex + 1) % state.getNumAgents()
        # 只有當下一個 agent 是 Pacman (代表一輪結束) 時，depth 才 +1
        if nextAgent == 0:
            nextDepth = depth + 1
        else:
            nextDepth = depth

        # Pacman 的回合 (MAX 節點)：選擇所有子節點中分數最大的
        if agentIndex == 0:
            bestValue = float('-inf') # 初始化為負無限大，確保任何值都能更新
            for action in state.getLegalActions(0): # 遍歷 Pacman 所有合法動作
                childState = state.getNextState(0, action) # 產生執行這個動作後的子狀態
                value = self.minimax(childState, nextAgent, nextDepth) # 這個子狀態傳給下一層遞迴
                bestValue = max(bestValue, value) # 保留最大值 (MAX 玩家選最好的)
            return bestValue
            # 如果我是 Pacman，我會選最好的那步，所以這個state的價值就是那個最大分數。

        # 鬼魂的回合 (MIN 節點)：選擇所有子節點中分數最小的
        else:
            bestValue = float('inf') # 初始化為正無限大，確保任何值都能更新
            for action in state.getLegalActions(agentIndex): # 遍歷該鬼魂所有合法動作
                childState = state.getNextState(agentIndex, action) # 產生執行這個動作後的子狀態
                value = self.minimax(childState, nextAgent, nextDepth) # 這個子狀態傳給下一層遞迴
                bestValue = min(bestValue, value) # 保留最小值 (MIN 玩家選最差的)
            return bestValue
            # 如果我是鬼魂，我會選讓 Pacman 最慘的那步，所以這個state的價值就是那個最小分數。

        # 子節點的 return bestValue → 變成父節點的 value → 父節點再拿去比較更新 bestValue
```

```

# getAction 是 Minimax 的頂層，為分數→動作的轉換層
# 裡面會呼叫 minimax 來評估每個動作的分數，最後回傳分數最高的動作執行
def getAction(self, gameState):
    bestAction = None # 記錄目前最佳動作，初始為 None (尚未找到)
    bestScore = float('-inf') # 記錄目前最高分數，初始為負無限大

    # 遍歷 Pacman 所有合法動作 (相當於 MAX 節點的頂層展開)
    for action in gameState.getLegalActions(0):
        childState = gameState.getNextState(0, action) # 產生 Pacman 執行動作後的子狀態
        score = self.minimax(childState, agentIndex=1, depth=0) # 從鬼魂回合開始遞迴 (depth=0)
        if score > bestScore: # 若此動作的分數更高，更新最佳動作
            bestScore = score
            bestAction = action

    return bestAction # 回傳分數最高的動作

```

Questions1 (5%):

In the initial state of the `minimaxClassic` layout, the Minimax evaluation values are a stable 9, 8, and 7 at search depths 1, 2, and 3, respectively. However, why does the evaluation value suddenly plummet to a dismal -492 at depth 4? What specific event is the algorithm foreseeing at depth 4 that causes this drastic drop?

Please execute the following command.

```
python pacman.py -p MinimaxAgent -l minimaxClassic -a depth=4
```

Why -492? What specific event?

Losing the game incurs a penalty of -500 points, combined with accumulated time penalties, resulting in a leaf node score of approximately -492.

At depths 1 to 3, the reachable states of ghosts within the search tree do not yet include a terminal state where Pacman is caught. However, since `minimaxClassic` is an extremely small layout, the expanded search tree contains states where a ghost can actually catch and eat Pacman by depth 4.

Since Minimax assumes ghosts always play optimally, as long as there exists any path where a ghost can kill Pacman, Minimax treats that outcome as inevitable. Therefore, the algorithm evaluates the current state as a guaranteed loss, causing the score to plummet to -492.

Depth 4 gives Minimax enough lookahead to discover that Pacman will inevitably be caught under optimal ghost play, so the evaluation value drops drastically.

Part 2: Alpha-Beta Pruning (9%)

Explain your implementation (4%):

```
class AlphaBetaAgent(MultiAgentSearchAgent):
    """
    Alpha-Beta Pruning Agent :
    在 Minimax 的基礎上加入剪枝機制，大幅減少需要展開的節點數，但結果與 Minimax 完全相同。

    - alpha: MAX 玩家 (Pacman) 目前已保證能拿到的最低分。
    - beta: MIN 玩家 (鬼魂) 目前已保證能壓到的最高分。
    - MAX 節點剪枝條件：當  $v > \beta$ ，代表鬼魂不會選這條路（已有更好的選擇），直接放棄。
    - MIN 節點剪枝條件：當  $v < \alpha$ ，代表 Pacman 不會選這條路（已有更好的選擇），直接放棄。
    - Pruning 使用不等號 ( $>$  和  $<$ )，不在等號時剪枝。
    - 子節點按照 getLegalActions 的原始順序處理，不能重新排序。
    """

    # alphabeta 是核心遞迴函式，邏輯與 Minimax 相同，但加入 alpha 和 beta 參數來進行剪枝
    def alphabeta(self, state, agentIndex, depth, alpha, beta):
        # 終止條件1：遊戲已結束（贏或輸），直接回傳評估分數
        if state.isWin() or state.isLose():
            return self.evaluationFunction(state)

        # 終止條件2：輪到 Pacman 且已達到最大搜尋深度，回傳葉節點評估分數
        if agentIndex == 0 and depth == self.depth:
            return self.evaluationFunction(state)

        # 計算下一個行動的 agent 與深度（邏輯與 Minimax 相同）
        nextAgent = (agentIndex + 1) % state.getNumAgents()
        if nextAgent == 0:
            nextDepth = depth + 1
        else:
            nextDepth = depth
```

```
# Pacman 的回合 (MAX 節點)
if agentIndex == 0:
    bestValue = float('-inf') # 初始化為負無限大
    for action in state.getLegalActions(0):
        childState = state.getNextState(0, action)
        # 傳遞 alpha, beta 給子節點，讓子節點能進行剪枝
        value = self.alphabeta(childState, nextAgent, nextDepth, alpha, beta)
        bestValue = max(bestValue, value)
        # beta剪枝：Pacman 所選的節點分數已高於鬼魂保證能壓到的最高分 (beta)，鬼魂不會選這條路
        if bestValue > beta:
            return bestValue
        alpha = max(alpha, bestValue) # 更新 alpha: Pacman 找到更好的選擇
    return bestValue

# 鬼魂的回合 (MIN 節點)
else:
    bestValue = float('inf') # 初始化為正無限大
    for action in state.getLegalActions(agentIndex):
        childState = state.getNextState(agentIndex, action)
        # 傳遞 alpha, beta 給子節點，讓子節點能進行剪枝
        value = self.alphabeta(childState, nextAgent, nextDepth, alpha, beta)
        bestValue = min(bestValue, value)
        # alpha剪枝：鬼魂所選的節點分數已低於 Pacman 保證能拿到的最低分 (alpha)，Pacman 不會選這條路
        if bestValue < alpha:
            return bestValue
        beta = min(beta, bestValue) # 更新 beta: 鬼魂找到更能壓低分數的選擇
    return bestValue
```

```

# getAction 是 Alpha-Beta 的頂層，為分數→動作的轉換層
# 裡面會呼叫 alphabeta 來評估每個動作的分數，最後回傳分數最高的動作執行
def getAction(self, gameState):
    bestAction = None
    bestScore = float('-inf')
    alpha = float('-inf') # 初始 alpha = -∞, Pacman 尚未找到任何保證分數
    beta = float('inf') # 初始 beta = +∞, 鬼魂尚未找到任何壓制分數

    # 頂層展開 Pacman 的所有合法動作 (相當於 MAX 節點)
    for action in gameState.getLegalActions(0):
        childState = gameState.getNextState(0, action)
        score = self.alphabeta(childState, agentIndex=1, depth=0, alpha=alpha, beta=beta)
        if score > bestScore: # 若此動作更好，更新最佳動作
            bestScore = score
            bestAction = action
        alpha = max(alpha, score) # 頂層也要更新 alpha，讓後續子節點能正確剪枝

    return bestAction

```

Questions 2 (5%):

Explain the differences between Alpha-Beta pruning and standard Minimax when performing a depth 3 search, and discuss why these differences occur.

Please execute the following command.

```
python pacman.py -p MinimaxAgent -a depth=3 -l minimaxClassic -q -n 100
```

```
python pacman.py -p AlphaBetaAgent -a depth=3 -l minimaxClassic -q -n 100
```

Standard Minimax vs Alpha-Beta Pruning

Average Score: -101.96	Average Score: -132.18
Win Rate: 39/100 (0.39)	Win Rate: 36/100 (0.36)
Total Time: 14.03 seconds	Total Time: 11.79 seconds

Analysis

1. The key difference is execution time (Total Time).

Alpha-Beta pruning runs approximately 16% faster than Minimax (14.03s → 11.79s).

This is due to the pruning mechanism:

- Alpha-pruning: When a MIN node finds a value already below Pacman's guaranteed score (alpha), the entire subtree is discarded.
- Beta-pruning: When a MAX node finds a value already above the ghost's guaranteed score (beta), the entire subtree is discarded.

By skipping branches that do not affect the final decision, Alpha-Beta pruning evaluates far fewer nodes while producing exactly the same action as Minimax for any given game state.

2. Win Rate and Average Score is not a fundamental difference.

The difference in win rate and average score between the two runs is not caused by algorithmic difference, as both agents choose identical actions given the same game state.

The variation comes from the ghosts moving randomly. Since the 100 games are run independently with different random outcomes, the statistics will fluctuate. Both agents would converge to the same win rate and average score over a larger sample.

The only fundamental difference between Alpha-Beta Pruning and standard Minimax is efficiency. Pruning allows Alpha-Beta to skip branches that do not influence the final result, reducing computation time, but the action chosen is always identical to what Minimax would select.

Part 3: Search (9%)

Explain your implementation (4%):

```
class ExpectimaxAgent(MultiAgentSearchAgent):
    """
    ExpectimaxAgent:
    改良自 Minimax，適合對抗「非最優對手」（例如隨機行動的鬼魂）。

    - Pacman (MAX 節點) 的邏輯與 Minimax 完全相同，選擇最大值。
    - 鬼魂改為「Chance 節點」，不再假設鬼魂選最壞的，而是假設鬼魂均勻隨機選動作。
    - Chance 節點的值 = 所有子節點分數的平均值（期望值）。
    - 因為鬼魂不是最優對手，Expectimax 的 Pacman 行為比 Minimax 更大膽。
    - 無法使用 Alpha-Beta Pruning (Chance 節點的期望值必須看完所有子節點才能確定)。
    """

    # expectimax 是核心遞迴函式，邏輯與 Minimax 相似
    # 但鬼魂改為 Chance 節點，計算所有子節點分數的平均值（期望值）
    def expectimax(self, state, agentIndex, depth):
        # 終止條件1：遊戲已結束（贏或輸），直接回傳評估分數
        if state.isWin() or state.isLose():
            return self.evaluationFunction(state)

        # 終止條件2：輪到 Pacman 且已達到最大搜尋深度，回傳葉節點評估分數
        if agentIndex == 0 and depth == self.depth:
            return self.evaluationFunction(state)

        # 計算下一個行動的 agent 與深度（邏輯與 Minimax 相同）
        nextAgent = (agentIndex + 1) % state.getNumAgents()
        if nextAgent == 0:
            nextDepth = depth + 1
        else:
            nextDepth = depth
```

```

# Pacman 的回合 (MAX 節點) : 與 Minimax 完全相同, 選最大值
if agentIndex == 0:
    bestValue = float('-inf') # 初始化為負無限大
    for action in state.getLegalActions(0):
        childState = state.getNextState(0, action)
        value = self.expectimax(childState, nextAgent, nextDepth)
        bestValue = max(bestValue, value) # 選擇分數最高的動作
    return bestValue

# 鬼魂的回合 (Chance 節點) : 計算所有子節點分數的平均值 (期望值)
else:
    totalValue = 0 # 累加所有子節點的分數
    actions = state.getLegalActions(agentIndex)
    if len(actions) == 0: # 若無合法動作, 回傳 0
        return 0
    for action in actions:
        childState = state.getNextState(agentIndex, action)
        value = self.expectimax(childState, nextAgent, nextDepth)
        totalValue += value # 累加每個動作的分數
    averageValue = totalValue / len(actions) # 除以動作數量, 得到期望值
    return averageValue

```

```

# getAction 是 Expectimax 的頂層, 為分數→動作的轉換層
# 裡面會呼叫 expectimax 來評估每個動作的分數, 最後回傳分數最高的動作執行
def getAction(self, gameState):
    bestAction = None # 記錄目前最佳動作
    bestScore = float('-inf') # 記錄目前最高分數

    # 遍歷 Pacman 所有合法動作 (頂層 MAX 節點展開)
    for action in gameState.getLegalActions(0):
        childState = gameState.getNextState(0, action)
        score = self.expectimax(childState, agentIndex=1, depth=0) # 從鬼魂回合開始遞迴
        if score > bestScore: # 若此動作的期望分數更高, 更新最佳動作
            bestScore = score
            bestAction = action

    return bestAction # 回傳期望分數最高的動作

```

Questions 3 (5%):

Why does the **ExpectimaxAgent** achieve an approximate 50% win rate on the **trappedClassic** layout, while the **AlphaBetaAgent** consistently loses 100% of the time under the exact same conditions?

Please execute the following command.

python pacman.py -p AlphaBetaAgent -l trappedClassic -a depth=3 -q -n 10
python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3 -q -n 10

AlphaBetaAgent vs ExpectimaxAgent

Average Score:	-501.0
Win Rate:	0/10 (0.00)
Total Time:	0.03 seconds

Average Score:	118.4
Win Rate:	6/10 (0.60)
Total Time:	0.11 seconds

Analysis

Why does AlphaBetaAgent lose 100% of the time?

The consistent score of -501 reveals exactly what happens: Pacman takes exactly one step (-1 time penalty), then immediately dies (-500), giving -501 every game.

AlphaBetaAgent is built on the assumption that ghosts always play optimally.

Looking 3 steps ahead, the minimax tree evaluates that all actions lead to the same inevitable death under optimal ghost play, every move has the same minimax value.

The agent simply picks the first action evaluated, walks directly into the ghost, and dies in one step every single game.

Why does ExpectimaxAgent achieve an approximately 50% win rate?

ExpectimaxAgent models ghosts as chance nodes, assuming they choose actions uniformly at random rather than optimally. The ghosts do in fact move randomly, making Expectimax's assumption much more realistic.

- Sometimes ghosts randomly move away: Pacman can dash out and win (+532)
- Sometimes ghosts happen to block the exit: Pacman gets caught (two steps before death) (-502)

Because Expectimax accounts for this randomness, it correctly identifies that moving away from the ghosts has positive expected value, and takes the risk rather than treating death as certain.

AlphaBetaAgent is overly pessimistic, as its worst-case assumption about ghost behavior is too strong, causing the agent to treat all moves as equally hopeless and walk directly into death. ExpectimaxAgent models the true behavior of the ghosts more accurately, resulting in more rational decision-making and achieving an approximately 50% win rate.

Part 4: Evaluation Function (9%)

Explain your implementation (4%):

```
# 調整各項評分權重，讓 Pacman 更積極地追食物，吃 capsule 製造 scared 狀態，同時更謹慎地避開鬼魂
def betterEvaluationFunction(currentGameState):
    """
    改良版的評估函式，評估當前遊戲狀態(而非動作)的優劣。

    設計考量：
    1. 食物距離：引導 Pacman 靠近食物，每顆食物都給予距離倒數的加分，最近食物再額外加分。
    2. 鬼魂處理：
        - scaredTimer > 5 (鬼魂仍處於 scared 狀態且時間充裕)：給予大量固定獎勵 (100分)，
          鼓勵 Pacman 維持讓鬼魂 scared 的狀態 (吃 capsule)，但不強迫追鬼以避免計時器歸零的風險。
        - 鬼魂正常且距離 <= 2：大幅扣分，距離越近扣越多，避免近距離危險。
        - 鬼魂正常且距離 > 2：輕微扣分，引導 Pacman 不要朝鬼魂方向移動。
    3. Capsule 距離：靠近能量豆加分，讓 Pacman 主動去吃 capsule 製造 scared 狀態。
    """
    # 基礎分數：當前遊戲狀態的分數 (包含已吃食物、時間懲罰、吃鬼獎勵等)
    score = currentGameState.getScore()

    # 取得 Pacman 目前位置
    pacPosition = currentGameState.getPacmanPosition()

    # 取得所有剩餘食物的座標列表
    findFood = currentGameState.getFood()
    foodList = []
    for x in range(findFood.width): # 遍歷地圖所有 x 座標
        for y in range(findFood.height): # 遍歷地圖所有 y 座標
            currentFood = findFood[x][y] # 取得該格是否有食物 (True/False)
            if currentFood == True: # 若有食物，記錄其座標
                foodList.append((x, y))

    # 食物距離評分：對每顆食物加上距離倒數，並額外獎勵最近的食物
    if foodList:
        nearestFood = float('inf') # 記錄最近食物距離，初始為正無限大
        for food in foodList:
            currentFood = manhattanDistance(pacPosition, food) # 計算到該食物的曼哈頓距離
            score += 3.0 / currentFood # 每顆食物都加分，距離越近加越多
            if currentFood < nearestFood: # 更新最近食物距離
                nearestFood = currentFood
        score += 6.0 / nearestFood # 對最近的食物額外加分，強化靠近行為

    # 鬼魂距離評分：根據 scaredTimer 分兩種情況處理
    ghostStates = currentGameState.getGhostStates()
    for ghost in ghostStates:
        ghostPosition = ghost.getPosition() # 取得鬼魂座標
        ghostScaredTime = ghost.scaredTimer # 取得 scared 剩餘時間
        ghostDistance = manhattanDistance(pacPosition, ghostPosition) # 計算與鬼魂的距離
        if ghostDistance > 0: # 避免除以零
            if ghostScaredTime > 5:
                score += 100.0 # scared 時間充裕：給予大量獎勵，鼓勵維持此狀態
                # ghostScaredTime 大於 5 時，給予大量分數獎勵，Pacman 可以安全地吃掉鬼魂
                # 但當小於 5 時，不再有獎勵，讓Pacman不要再靠近鬼魂
            elif ghostDistance <= 2:
                score -= 2.0 / ghostDistance # 正常鬼魂且非常近：大幅扣分，緊急避開
            else:
                score -= 1.5 / ghostDistance # 正常鬼魂但距離較遠：輕微扣分，保持警覺

    # Capsule 距離評分：靠近能量豆加分，引導 Pacman 去吃 capsule 以製造 scared 狀態
    capsuleList = currentGameState.getCapsules()
    for capsule in capsuleList:
        capsuleDistance = manhattanDistance(pacPosition, capsule) # 計算到能量豆的距離
        score += 1.0 / capsuleDistance # 距離越近加越多分

    return score # 回傳綜合評估分數
```

```
# 食物距離評分：對每顆食物加上距離倒數，並額外獎勵最近的食物
if foodList:
    nearestFood = float('inf') # 記錄最近食物距離，初始為正無限大
    for food in foodList:
        currentFood = manhattanDistance(pacPosition, food) # 計算到該食物的曼哈頓距離
        score += 3.0 / currentFood # 每顆食物都加分，距離越近加越多
        if currentFood < nearestFood: # 更新最近食物距離
            nearestFood = currentFood
    score += 6.0 / nearestFood # 對最近的食物額外加分，強化靠近行為

# 鬼魂距離評分：根據 scaredTimer 分兩種情況處理
ghostStates = currentGameState.getGhostStates()
for ghost in ghostStates:
    ghostPosition = ghost.getPosition() # 取得鬼魂座標
    ghostScaredTime = ghost.scaredTimer # 取得 scared 剩餘時間
    ghostDistance = manhattanDistance(pacPosition, ghostPosition) # 計算與鬼魂的距離
    if ghostDistance > 0: # 避免除以零
        if ghostScaredTime > 5:
            score += 100.0 # scared 時間充裕：給予大量獎勵，鼓勵維持此狀態
            # ghostScaredTime 大於 5 時，給予大量分數獎勵，Pacman 可以安全地吃掉鬼魂
            # 但當小於 5 時，不再有獎勵，讓Pacman不要再靠近鬼魂
        elif ghostDistance <= 2:
            score -= 2.0 / ghostDistance # 正常鬼魂且非常近：大幅扣分，緊急避開
        else:
            score -= 1.5 / ghostDistance # 正常鬼魂但距離較遠：輕微扣分，保持警覺

# Capsule 距離評分：靠近能量豆加分，引導 Pacman 去吃 capsule 以製造 scared 狀態
capsuleList = currentGameState.getCapsules()
for capsule in capsuleList:
    capsuleDistance = manhattanDistance(pacPosition, capsule) # 計算到能量豆的距離
    score += 1.0 / capsuleDistance # 距離越近加越多分

return score # 回傳綜合評估分數
```

Questions 4 (5%):

What specific factors are incorporated into your custom Evaluation Function, why are they fundamentally important for guiding the agent, and how do they collectively contribute to a win rate of more than half the time?

Please execute the following command.

```
python pacman.py -p ExpectimaxAgent -l smallClassic -q -n 10
```

```
python pacman.py -p ExpectimaxAgent -a evalFn=better -l smallClassic -q -n 10
```

Original Evaluation Function vs Custom Evaluation Function

Average Score: 272.2	Average Score: 1399.1
Win Rate: 3/10 (0.30)	Win Rate: 10/10 (1.00)
Total Time: 28.33 seconds	Total Time: 8.29 seconds

Custom Evaluation Function Design and Analysis

The betterEvaluationFunction incorporates four key factors:

1. Food Distance

The default scoreEvaluationFunction only considers the current game score, providing no incentive for Pacman to actively approach food.

However, the betterEvaluationFunction makes closer food contribute more to the score, guiding Pacman to move toward food rather than wandering aimlessly. This significantly reduces the time needed to clear the board.

2. Ghost Distance

I make ghosts be handled in three distinct cases, allowing Pacman to aggressively collect food when safe and immediately evade when threatened, without taking unnecessarily long detours to avoid ghosts.

3. Capsule Distance

I encourage Pacman to actively seek out power capsules, triggering the scared state of ghosts. With ghosts temporarily harmless, Pacman can collect food more safely and even get bonus points.

4. Base Game Score

The original game score is retained as a foundation, ensuring Pacman still values its core objectives like eating food for points and avoiding time penalties.

Why is the betterEvaluationFunction so effective?

The default scoreEvaluationFunction only reflects the current score, so Expectimax has no way to recognize the benefit of moving toward food. This causes Pacman to act slowly, take suboptimal routes, and frequently encounter ghosts.

However, the betterEvaluationFunction provides far richer state information, allowing Expectimax to make more informed decisions at every step. The result is a dramatic improvement: win rate jumps from 30% to 100%, and the average score increases by more than 5 times higher than the original, while execution time also drops by more than half due to faster and more decisive play.

Question 5: AI (Mis)Alignment and Reward Hacking (4%)

According to question 3, Pacman's behavior above is an example of one **concrete problem in AI alignment** called **reward hacking**, which occurs when an agent satisfies some objective but may not actually fulfill the designer's intended goals, due e.g. to an imprecise definition of the objective function. As another example, a cleaning robot rewarded for minimizing the number of messes in a given space could optimize its reward function by hiding the messes under the rug. In this case, the agent finds a shortcut to optimize the reward, but the shortcut fails to attain the designer's goals (see [this list](#) for more examples).

Even if the agent *does* satisfy the designer's goals, another problem can arise (again see [this paper](#)): the agent's behavior might cause **negative side effects** that come in conflict with broader values held by society or other stakeholders. For instance, a social media content recommendation system might aim to maximize user engagement, but in doing so, spread disinformation and conspiracy theories (since such posts get the most engagement), which is at odds with societal values.

Give one example of either reward hacking or negative side effects caused by an AI agent. Explain why your example illustrates the problem.

Reward Hacking 的例子: CoastRunners

在 OpenAI 與學界的 RL 研究中, 其中有一個案例是 AI 被訓練在 CoastRunners 遊戲中想辦法獲得最高分。在遊戲地圖中散布著可以加分的獎勵物, 設計者原本假設 AI 會完成賽道、超越對手且快速抵達終點奪冠。

然而, AI 在過程中卻發現一個漏洞, 也就是靠近牆壁時可以反覆撞擊浮標取得分數, 不需完成賽道、不需比賽且風險很低。因此, AI 認為與其繞完整個賽道, 不如在一小段水域中反覆繞圈, 持續撞上同一批獎勵物刷分, 最終 AI 達到了極高的分數, 卻從未真正完成比賽。設計者真正的目標是贏得比賽, 但形式化目標卻是分數最大化, 其中出現了 Specification Gap (規格漏洞)。

為何 **CoastRunners** 事件符合 **Reward Hacking**：

AI 完美優化了設計者定義的獎勵函數(分數最大化)，但完全違背設計者的真實目標(完成比賽)。主要的問題在於獎勵函數的定義不夠精確，設計者沒有明確告訴 AI 完成賽道是必要條件，只告訴它分數越高越好。因此 AI 沒有真正完成「人類想要的目標」，而是找到一條「獲得高分但違背本意」的捷徑，這恰好呈現 Reward Hacking 的核心問題。